



Java Classes and Objects Summary

In Java, everything is centered around **objects** and **classes**. A class is the fundamental building block of a Java application.

1. What is a Class?

A **Class** is a **blueprint** or a template for creating objects. It defines the structure (the data/fields) and the behavior (the actions/methods) that all objects of that type will have.

An **Object** (also called an **instance**) is a concrete realization of that class.

Concept	Analogy	Java Terminology
Class	The blueprint for a house.	Defines the structure and methods.
Object	The actual house built from the blueprint.	A usable instance in memory.

2. Key Components of a Class

A typical Java class consists of three main parts:

A. Fields (Attributes/Data)

These are variables that hold the state or characteristics of an object.

Term	Example in a Car class
Field	String color; (State)
Field	int speed; (State)

B. Methods (Behavior/Actions)

These are functions that define what an object can do.

Term	Example in a Car class
------	------------------------

Method	public void accelerate() (Behavior)
Method	public String getColor() (Behavior)

C. Constructor

A special method used to initialize new objects. It has the same name as the class and runs when the new keyword is used.

3. Class Syntax and Object Creation

Step 1: Defining the Class

```
// The Class Definition (The Blueprint)
public class Dog {
    // A. Fields/Attributes
    String name;
    int age;

    // C. Constructor: Initializes a new Dog object
    public Dog(String dogName, int dogAge) {
        this.name = dogName; // 'this' refers to the object being created
        this.age = dogAge;
    }

    // B. Method: Defines a behavior
    public void bark() {
        System.out.println(this.name + " says Woof!");
    }
}
```

Step 2: Creating an Object (Instantiation)

To use the class, you must create an object from it using the new keyword.

```
public class Kennel {
    public static void main(String[] args) {
        // Creates a new Dog object named 'myDog' using the constructor
        Dog myDog = new Dog("Buddy", 3);

        // Accessing the object's method
        myDog.bark();
    }
}
```

```
// Accessing the object's field (state)
System.out.println(myDog.name + " is " + myDog.age + " years old.");
}
}
```